PoE-agent: Automated Exploits Generation for Smart Contract Vulnerabilities Management

Cheng Ning
CUNY Graduate Center
cheng.ning29@brooklyn.cuny.edu

Hui Chen
CUNY Brooklyn College
hui.chen@brooklyn.cuny.edu

Abstract

Demonstrating that a smart contract vulnerability is exploitable typically requires constructing a vulnerability Proof-of-Concept (PoC). In practice, creating such executable exploits is a largely manual process that demands significant expertise, careful reasoning about program, and extensive trial-and-error, especially in a concrete execution environment.

To bridge the gap, we present PoE-agent, an agentic framework for automatically generating executable Proofs of Exploitation (PoE) from vulnerability reports and smart contract codebases. PoE extends PoC by including an PoE specification and a runner for simulated execution. PoE-agent incrementally recovers missing context using security analysis tools, synthesizes a structured PoE specification, and iteratively revises it using execution feedback until it becomes executable and reproduces the reported security violation.

We evaluate PoE-agent on the PoE-bench smart contract dataset, a collection of 56 real-world vulnerabilities spanning 17 categories that we curated from a bug bounty platform. Excluding 1 true out-of-scope instance, PoE-agent achieves a 43.6% success rate (24/55), substantially reducing the manual effort required for vulnerability validation and exploit development.

1 Introduction

Smart contracts represent computer programs or network protocols that automatically executes predefined rules governing an agreement regarding digital assets or processes [14]. Recent advancement of blockchain technology have enabled practical implementations of smart contracts in which predefined conditions are verified through on-chain data and trusted oracle inputs [14]. Owing to their benefit of reducing or eliminating the need for manual enforcement by intermediaries, smart contracts has gained adoption across blockchain platforms [19]. For instance, millions of smart contracts have been deployed to the Ethereum blockchain alone [2]. However, because smart contracts often control valuable digital assets, vulnerabilities in their implementation can lead to irreversible financial losses. Thus, ensuring the security of smart contracts has become a critical challenge for the reliable deployment of blockchain-based applications.

Blockchain smart contracts are typically *immutable* once deployed, as such, mitigating the impact of vulnerabilities is challenging after deployment. This immutability shifts the security burden decisively to the *pre-deployment phase*, making rigorous vulnerability management essential. As in broader software security practice, vulnerability management follows a structured lifecycle consisting of (1) *vulnerability assessment*, (2) *vulnerability validation and prioritization*, and (3) *vulnerability resolution* [12]. Among these stages, vulnerability validation is critical for smart contracts because it determines whether a reported security issue truly corresponds to an exploitable condition. The accurate validation avoids unnecessary remediation effort while ensuring genuine security risks are not overlooked [13]. In practice, vulnerability validation is commonly performed using artifacts referred to as vulnerability *Proofs of Concept* (PoCs). A PoC demonstrates that a reported vulnerability can be triggered under realistic conditions and are widely used in security audit and vulnerability reports [18]. For smart contracts, PoCs typically take the form of exploit transactions or scripts that reproduce the vulnerable behavior on a test network or execution environment. However, constructing such PoCs often requires substantial manual effort and deep domain expertise. This very challenge motivates our research on automated techniques for generating PoC exploits for smart contract vulnerabilities.

Recent work has shown that generative Large Language Models (LLMs) and agentic workflows can automate the generation of code as PoC for smart contracts [1, 3, 7, 21]. First, these approaches typically operate on individual contracts, despite the fact that smart contracts are rarely standalone entities. In practice, they require a network and platform to execute and often interact with other contracts. Reasoning about a single contract in isolation misses critical project-level semantics and dependencies. Second, their evaluations are

commonly restricted to curated datasets or narrow vulnerability classes. Third, the integration of security-oriented analysis tools and systematic context management of agent remains underexplored.

To address these limitations, we introduce a framework that automatically generates executable PoEs from vulnerability reports and smart contract repositories using LLMs. The framework is characterized with a formal notion of a PoE and a PoE runner. A PoE provides a precise machine-checkable definition of vulnerability validation. it defines (i) the preconditions under which the intended security property still holds (ii) a transaction sequence that triggers the vulnerability (iii) an attestation of the violation of an intended security property. The PoE runner prepares a simulation environment and carries out the exeuction sequences as specified in a PoE automatically. Thus, the framework not only validates the existence of a vulnerability but also significantly reduces, if not completely eliminates, the manual labor required for the validation process. Additionally, the design from PoE-agent stems from a formal analysis of the problem and the associated modeling approach. This analysis leads to a principled design that incrementally recovers missing context using static security analysis tools, synthesizes a structured PoE specification, and iteratively revises it using execution feedback until it becomes executable and reproduces the reported security violation.

We summarize the contributions of this work as follows:

1. *A formal foundation for smart contract vulnerability validation.* We introduce the notion of a **Proof of Exploitation (PoE)**—a machine-checkable specification that captures explicit preconditions, transaction traces, and security property violation attestation. This provides, for the first time, a precise definition of machine-checkable proof for reported vulnerability, enabling clear and systematic vulnerability validation.

2. *An agentic framework for automated PoE generation.* We present **PoE-agent (POET)**, which combines static program analysis with reasoning about execution feedback to generate PoEs from vulnerability reports and smart contract repositories. The framework demonstrates how principled problem analysis, rather than ad-hoc prompting, can guide the design of LLM-based agentic security systems.

3. *An agentic runtime sandbox for automated PoE generation.* We build **PoE-Runtime (POER)**, a sandboxed execution environment enable agent can iteratively analyze contracts, refine the PoE specification, execute candidate transaction traces, and observe execution results. This closed-loop interaction enables automated construction and validation of exploits in a controlled environment.

4. *A benchmark for evaluating automated exploit generation.* We release **PoE-bench**, a collection of real-world vulnerable smart contracts sourced from bug bounty platforms. This benchmark enables systematic evaluation of automated PoE generation techniques on authentic vulnerabilities.

2 Background

This section is aimed to provide a background knowledge for blockchain smart contracts and for AI Agentic systems.

2.1 Blockchain concepts

Although it can be extended to other blockchain systems, the proposed PoE-agent is implemented for Ethereum smart contracts. We briefly review standard Ethereum concepts and terminology used in this paper, following the Ethereum Yellow Paper [20]. Readers familiar with Ethereum may skip this section.

External Actor A person or other entity able to interface to an Ethereum node, but external to the world of Ethereum. It can interact with Ethereum through depositing signed transactions (defined below) and inspecting the blockchain and associated state. An external actor has one (or more) intrinsic accounts (also defined below).

Account and Address Ethereum defines state objects called "accounts", with each account having a unique 20-byte, i.e., 160-bit address. There are two types of accounts: externally owned accounts (EOA) and contract accounts. The former also called an intrinsic account is specified by a private key. A contract account is an account that is controlled by a contract code. Accounts can serve as senders and targets of transactions.

Transaction In Ethereum, transactions represent direct transfers of value and information between accounts. It can be either a Message (Message-call transaction) or a new Autonomous Object(contract-create transacton). Transactions are signed by an external actor and are recorded into each block of the blockchain.

EVM Code The bytecode that the Ethereum Virtual Machine (EVM) can natively execute. Used to formally specify the meaning and ramifications of a message to an Account.

Contract Informal term used to mean both a piece of EVM Code that may be associated with an Account or an Autonomous Object.

Autonomous Object A notional object existent only within the hypothetical state of Ethereum. Has an intrinsic address and thus an associated account; the account will have non-empty associated EVM Code. Incorporated only as the Storage State of that account.

State Transition System Blockchain systems can be abstracted as deterministic state transition systems. Formally, we model a blockchain platform by a deterministic state transition function:

$$Y(\sigma_t, \vec{T}) = \sigma_{t+1} \tag{1}$$

where $\sigma_t \in \Sigma$ is a valid pre-state, $\vec{T} \in \mathcal{T}^*$ is a finite sequence of valid transactions, and $\sigma_{t+1} \in \Sigma$ is the resulting post-state. Here, Σ denotes the set of all reachable blockchain states, and $\mathcal{T}^*$ denotes the set of all finite transaction sequences.

The transition function

$$Y : \Sigma \times \mathcal{T}^* \to \Sigma$$

is deterministic: for any given state σ_t and transaction sequence $\vec{T}$, the resulting state σ_{t+1} is uniquely determined.

Message Data (as a set of bytes) and Value (specified as Ether) that is passed between two Accounts, either through the deterministic operation of an Autonomous Object or the cryptographically secure signature of the Transaction.

Message call The act of passing a message from one Account to another. If the destination account is associated with non-empty EVM Code, then the VM will be started with the state of said Object and the Message acted upon. If the message sender is an Autonomous Object, then the Call passes any data returned from the VM operation.

2.2 Agentic system

An agentic system refers to a system that operates along a spectrum of agency, characterized by its ability to plan, use external actions, reflect on intermediate states, and carry out iterative multi-step behaviors in pursuit of a given objective, rather than producing a single one-shot response [16, 23, 24].

Agentic designs have been shown to be effective for tasks that require multi-step reasoning, long-horizon planning, and environment interaction, including software engineering and program analysis [22]. Compared to single-shot prompting, agentic workflows explicitly expose intermediate decision points and allow reasoning to be grounded through feedback from external execution or analysis tools.

3 Problem Definition

Given a set of vulnerability report R and a collection of smart contracts $\mathcal{D}$, our objective is to generate executable PoEs that concretely demonstrate the reported vulnerabilities.

To clarify the problem setting, we first illustrate a running example, and then formalize the task.

Vulnerable smart contract A vulnerable smart contract is a contract whose implementation violates an intended security property under certain execution conditions. Such violations typically arise from incorrect state updates, improper access control, unsafe external calls, and other implementation errors.

Listing 1: Smart contract with a "reentrancy" vulnerability. The contract performs an external call before updating internal state, violating the checks-effects-interactions pattern.

```
pragma solidity ^0.8.20;

contract SimpleVault {
    mapping(address => uint256) public balance;

    function deposit() external payable {
        balance[msg.sender] += msg.value;
    }

    // BUG: external call before state update
    function withdraw(uint256 amount) external {
        require(balance[msg.sender] >= amount);
        (bool ok,) = msg.sender.call{value: amount}("");
        require(ok);
        balance[msg.sender] -= amount;
    }
}
```

Listing 1 shows a simplified smart contract example. The contract `SimpleVault` allows users to deposit and withdraw Ether, i.e., Ethereum cryptocurrency (ETH). However, the `withdraw` function performs an external call to `msg.sender` before updating the internal balance mapping. This violates the checks-effects-interactions pattern and introduces a reentrancy vulnerability, which can results in so called "double spend" as explained below.

Exploiting vulnerabilities Exploiting a smart contract vulnerability means constructing and executing a sequence of transactions that drives the contract into a state violating its intended security property.

In Ethereum, all state changes occur through transactions. Therefore, an exploit is fundamentally an ordered transaction sequence $\vec{T} = (T_1, \ldots, T_n)$ such that resulting state violates the security property. For the reentrancy vulnerability shown in Listing 1, the exploit sequence is:

- T_1: A contract-creation transaction that deploys the attacker contract (Listing 2) with reentrant runtime code.
- T_2: A message-call transaction invoking `seed()` to establish a positive balance in the vulnerable contract.
- T_3: A message-call transaction invoking `attack()`, which triggers the first `withdraw`.

The necessity of T_1 follows from Ethereum's execution model. Unlike externally owned accounts (EOAs), contract accounts contain runtime bytecode. When a contract transfers Ether to another contract, the EVM transfers control to the recipient's fallback entry point, in this case, the `receive()` function, within the same transaction.

Listing 2: Example exploiting the "reentrancy" vulnerability in `SimpleVault.withdraw`. Via it, an attacker can deposit once, trigger `withdraw`, and reenter via `receive()` another `withdraw` before the victim updates internal state.

```
pragma solidity ^0.8.20;

interface ISimpleVault {
    function deposit() external payable;
    function withdraw(uint256 amount) external;
}

contract ReenterExploit {
    ISimpleVault public vault;
    uint256 public unit; // withdraw unit each
        reentry

    constructor(ISimpleVault v) { vault = v; }

    // Step 1: seed vault with collateral from
        attacker contract
    function seed() external payable {
        vault.deposit{value: msg.value}();
        unit = msg.value; // simplest: withdraw full
            deposit each time
    }

    // Step 2: trigger the first withdraw,
        reentrancy happens in receive()
    function attack() external {
        vault.withdraw(unit);
    }

    // Reenter before SimpleVault updates balance
        [msg.sender]
    receive() external payable {
        // keep draining while vault still has ETH
        if (address(vault).balance >= unit) {
            vault.withdraw(unit);
        }
    }
}
```

During T_3, the vulnerable contract executes a value-transferring call to the attacker contract before updating its storage. This transfer activates the attacker's `receive()` function. Because execution remains within the same transaction and the original state update has not yet occurred, the attacker can invoke `withdraw` again from within `receive()`.

Each reentrant invocation therefore observes the same pre-update state. Multiple withdrawals succeed against a single recorded balance, resulting in a double-withdrawal condition and draining Ether belonging to other users.

Vulnerability Reports In practice, vulnerability reports sometimes do not provide executable artifacts, but as natural-language reports written by auditors or security researchers. Listing 3 shows an example of high-severity report. Such reports describe the root cause, impact, and an example attack flow, but they rarely specify a complete, directly executable transaction sequence. Important details, such as precise parameters, required state initialization, or auxiliary contract deployments, are often omitted.

To formalize the PoE generation task, we abstract a vulnerability report R as a tuple:

$$R = (\sigma_t, \vec{T}_r, \sigma_{t+1}, \varphi) \tag{2}$$

where:

- $\sigma_t \in \Sigma$ is the initial world state, i.e., the complete world state prior to executing the exploit transaction sequence, consisting of all accounts and their associated state (including balance, nonce, contract code, and storage).
- $\vec{T}_r = (T_1, T_2, \ldots, T_n) \in \mathcal{T}^*$ is the ordered reported transaction sequence describing the PoE exploit. $\vec{T}_r$ may omit transactions, parameters, or fields, and is not required to be directly executable. Each T_i may represent either a contract creation or a function call.
- $\sigma_{t+1} \in \Sigma$ is the final world state after executing a complete transaction sequence, where $\sigma_{t+1} = Y(\sigma_t, \vec{T})$ and Y is the state transition function. Here, $\vec{T}$ denotes a complete transaction sequence derived from $\vec{T}_r$.
- φ is a proposition accepted as true by external actors, representing the intended security property that is violated after executing the transaction sequence. Formally, we define φ using epistemic logic as $Accepted_G(\varphi) := \forall i \in G,\ K_i(\varphi)$, where G denotes the set of external actors who interact with or adopt the target contract, and $K_i(\varphi)$ indicates that actor i knows or believes that φ should hold in the post-execution state σ_{t+1}.

Target Repository Smart contract vulnerabilities often involve interactions among multiple contracts rather than a single contract in isolation.

We define the *target repository* as the smart contract codebase provided as input to the PoE generation task. Let $\mathcal{D}$ denote the target repository:

$$\mathcal{D} = \{C_1, C_2, \ldots, C_m\},$$

where each C_i is a smart contract contained in the repository.

The target repository comprises:

- the contract(s) that implement the vulnerable logic described in the vulnerability report
- all additional contracts within the same repository that are required to deploy, initialize, or interact with the vulnerable contract during exploit execution.

Proof of Exploitation While a vulnerability report describes the attack logic informally, a PoE should ideally demonstrates the vulnerability. As discussed in the introduction, the term *PoC* is widely used but often overloaded,

Listing 3: Example high-severity reentrancy report written in the style of competitive audit platforms.

```
[H-01] SimpleVault.withdraw allows reentrancy

Submitted by alice_eth

Impact

An attacker can withdraw more ETH than their deposited
    balance, draining the vault.

Proof of Concept

Users deposit ETH into SimpleVault via deposit().

SimpleVault.sol#L10-L22

The withdraw() function sends ETH to msg.sender before
    updating the internal balance mapping.

Because the balance is reduced only after the external
    call, an attacker contract can reenter withdraw()
    from its receive() function and repeatedly withdraw
    funds before the balance is decreased.

Example attack flow:

1) Attacker deposits 1 ETH.
2) Attacker calls withdraw(1 ETH).
3) Fallback reenters withdraw() before state update.
4) Multiple withdrawals succeed.

Recommended Mitigation Steps

Update state before external calls (checks-effects-
    interactions), or add a reentrancy guard modifier.
```

Listing 4: The `test_exploit` function demonstrates the proof of exploitation of the "reentrancy" vunerability.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import "../src/SimpleVault.sol";
import "../src/ReenterExploit.sol";

contract ReentrancyPoC is Test {

    function test_exploit() public {
        SimpleVault vault = new SimpleVault();
        ReenterExploit attacker = new ReenterExploit(vault
            );

        // Victim funds the vault
        vm.deal(address(0xBEEF), 10 ether);
        vm.prank(address(0xBEEF));
        vault.deposit{value: 10 ether}();

        // Attacker starts with 1 ETH
        vm.deal(address(attacker), 1 ether);

        uint256 before = address(attacker).balance;

        attacker.seed{value: 1 ether}();
        attacker.attack();

        uint256 afterBal = address(attacker).balance;

        assertGt(afterBal, before); // attacker
            profits
    }
}
```

and PoCs for smart contract vulnerabilities are insufficiently defined.

Motivated by this observation, following the Ethereum state transition model, we define an executable PoE as:

$$PoE = (\phi, Y(\sigma_t, \vec{T}), \psi) \tag{3}$$

where:

- Y is the deterministic state transition function.
- $\sigma_t \in \Sigma$ denotes the initial world state under test.
- $\phi : (\mathcal{S}, \mathcal{P}) \to \{\text{true}, \text{false}\}$ is a precondition predicate that specifies the constraints under which the exploit is applicable, including assumptions on the initial state and attacker-controlled parameters.
- $\vec{T} = (T_1, T_2, \ldots, T_n) \in \mathcal{T}^*$ is an ordered sequence of transactions, where each T_i represents either a contract creation or a function call.
- $\psi : (\mathcal{S}, \mathcal{P}) \to \{\text{true}, \text{false}\}$ is a postcondition predicate that evaluates whether the execution results in a violation of an intended security property.

Intuitively, ϕ captures the required setup conditions, $\vec{T}$ captures the concrete exploit steps, and ψ verifies that the attack succeeds.

As an example, Listing 4 illustrates a `test_exploit` function that demonstrates the existence of the reentrancy vulnerability illustrated in Listing 1. Under our definition, this test function constitutes a valid PoE. The function carries out a sequence of transactions: (1) deploys the vulnerable contract and the attacker contract, (2) funds the vault (satisfying ϕ), (3) executes the exploit transaction sequence $\vec{T}$, and (4) finally asserts that the attacker's balance increases, thereby satisfying the postcondition ψ.

Furthermore, our definition does not require PoEs to be tied to a specific testing framework or implementation. As discussed in Section 4.2, a PoE in this work is represented as a JSON object.

4 Design of PoE-agent

PoE-agent addresses the problem of automatically generating an executable PoE from a vulnerability report. Given a report R and a target repository $\mathcal{D}$, the objective is to synthesize a

PoE including a complete transaction sequence $\vec{T}$ such that executing $Y(\sigma_t, \vec{T})$ transitions the system into a state σ_{t+1} that violates the security property φ.

Given current research, this task is fundamentally difficult for following reasons:

1. **Incomplete Transaction Sequence.** Vulnerability reports rarely specify a complete exploit transaction sequence. Essential steps such as contract deployment, state initialization, prerequisite function calls, or implicit configuration assumptions are often omitted and must be inferred from the repository context.

2. **Ordered Multi-Step and Multi-Role Execution.** Exploits require a precisely ordered sequence of actions involving multiple interacting roles (e.g., attacker, victim, admin, auxiliary contracts). Violating ordering constraints or role assignments can prevent the system from reaching the intermediate states necessary for exploitation, invalidating the PoE.

3. **Cross-Contract Interactions.** Even when all required actions are known, they may span multiple contracts. Real-world exploits rely on interactions across contracts with distinct interfaces and lifecycles (e.g., ERC20 tokens, AMMs, governance modules), which must be correctly identified, deployed, and made callable within a unified execution context.

Table 1 summarizes the primary challenges and their corresponding architectural solutions.

4.1 Overview

To address above challenges, PoE-agent adopts an agentic framework that iteratively acquires missing information for reasoning, generate a PoE specification, and revise the PoE specification based on execution feedback until the PoE is executable and demonstrates the reported vulnerability. Figure 1 illustrates this workflow with a simplified example.

Conceptually, We model PoE-agent as a discrete-time partially observable Markov decision process (POMDP).

$$(\mathcal{S}, \mathcal{A}, O, \mathbb{T}, \mathbb{T}_0, \mathbb{O}, R, N).$$

- $\mathcal{S}$ is the unobservable environment state, A state s_t encodes the repository $\mathcal{D}$ and the PoE specification (initially empty).

- $\mathcal{A}$ denotes the structured action space. Actions include repository-level, PoE-level, progress-control actions

- O denotes the observation space, consisting of structured outputs returned by tools (e.g. code snippets, static-analysis result, execution feedback)

- $\mathbb{T}(s_{t+1} \mid s_t, a_t)$ is the state transition kernel. It defines environment evolution after an action (e.g. PoE edits)

- $\mathbb{T}_0$ is the initial state distribution. The initial state corresponds to the target repository with an empty PoE.

- $\mathbb{O}(o_t \mid s_t)$ is the observation kernel, defining the probability of receiving observation o_t given latent state s_t, capturing partial observability.

- $R : \mathcal{S} \to \mathbb{R}$ is a sparse episodic reward. A positive reward is assigned only when executing $\vec{T}$ yields a state σ_{t+1} that violates the security property φ.

- $N \in \mathbb{N} \cup \{+\infty\}$ denotes the reasoning horizon.

Policy. Because the environment is partially observable, actions are conditioned on the interaction history rather than the latent state. Let

$$\tau_t = (o_0, a_0, o_1, \ldots, a_{t-1}, o_t)$$

denote the trajectory up to time t. We define a history-dependent policy

$$\pi : \mathcal{H} \to \mathcal{A}, \quad a_t = \pi(\tau_t),$$

where $\mathcal{H}$ is the space of finite trajectories.

Algorithm 1 illustrate the whole PoE-agent process modeled as POMDP above. The process begins with the repository $\mathcal{D}$ and an empty PoE specification, while the vulnerability report R serves as the initial observation. At each step, the policy selects an action based on the accumulated trajectory.

The process starts from the initial state

$$s_0 = (\mathcal{D}, \mathrm{PoE}_0 = \emptyset) \sim \mathbb{T}_0,$$

where $\mathcal{D}$ is the target repository and the PoE specification is empty. The initial observation is the vulnerability report $o_0 = R$. The trajectory is initialized as $\tau_0 = (o_0)$.

At each step t, the history-dependent policy

$$a_t = \pi(\tau_t)$$

selects an action from the structured action space $\mathcal{A}$. The policy is implemented by a LLM under a fixed system prompt.

Retrieve missing information When the trajectory indicates that relevant contextual information is missing, the policy selects repository-level actions. These actions query source code, resolve contract dependencies, or retrieve static-analysis results. The environment transitions according to $\mathbb{T}$ and produces structured observations via $\mathbb{O}$.

PoE construction. Once sufficient contextual information has been gathered, the policy selects PoE-level construction actions to synthesize an initial PoE specification contained in s_t. These actions define the transaction sequence $\vec{T}$, actors, and required parameters. Formally, such actions modify the PoE component of the latent state via the transition kernel

$$s_{t+1} \sim \mathbb{T}(\cdot \mid s_t, a_t).$$

Table 1: Challenges in automated PoE generation and the corresponding design solutions in POE-AGENT.

Category	Challenge	Solution
PoE	Incomplete transaction sequences	Repository-level actions for investigation(Section 4.3.1)
	Ordered multi-step execution involving multiple roles	Explicit PoE specification, feedback-driven iterative refinement(Sections 4.2 and 4.3.2)
	Cross-contract interactions	Static-analysis–guided dependency resolution (Section 4.3.1)
LLM	Hallucinated assumptions	Repository actions for grounded inspection(Section 4.3.1)
	Repository information retrieval with token limits	Static-analysis integration to identify relevant contracts and functions (Section 4.3.1)
	Long-context forgetting	Progress-control actions to track and update reasoning state (Section 4.3.3)
	Extraneous source-code generation and boilerplate overhead	Structured PoE specification (Section 4.2)
	Unsafe modifications to the repository	Isolated PoE specification with execution inside the POER sandbox (Sections 4.2 and 5.1)
	Difficulty debugging failed exploit attempts	Structured runner execution feedback for iterative refinement (Section 4.2)

Execution and refinement. The policy then selects execution actions that submit the constructed transaction sequence $\vec{T}$ to the EVM. Execution induces a blockchain-level state transition and produces runtime feedback (e.g., traces, revert reasons, balance changes), which constitutes the next observation

$$o_{t+1} \sim \mathbb{O}(\cdot \mid s_{t+1}).$$

This feedback is appended to the trajectory and conditions subsequent decisions. Based on the new observation, the policy may select refinement actions to edit $\vec{T}$, adjust parameters, or insert missing steps. These refinement edits again update the PoE component of the state via $\mathbb{T}$.

If the selected action is TERMINATE, the loop ends. Termination corresponds to reaching a terminal state under the reward function R, which occurs when:

(i) executing the constructed transaction sequence $\vec{T}$ drives the EVM from σ_t to σ_{t+1} such that the security property φ is violated (successful PoE), or

(ii) the vulnerability report is determined to be out of scope.

Otherwise, the interaction continues until a terminal condition is reached or the reasoning horizon N is exceeded.

We next ground the above POMDP abstraction by instantiating its key components in the context of PoE-agent. Section 4.2 introduces the PoE specification and its execution semantics, thereby concretizing the state space $\mathcal{S}$. Subsequently, Section 4.3 defines the structured action space $\mathcal{A}$ of PoE-agent. The outcomes of these actions constitute the observation space O.

Together, these sections operationalize the abstract POMDP formulation in a concrete and executable framework.

Algorithm 1 POE-AGENT as a POMDP

POMDP: $(\mathcal{S}, \mathcal{A}, O, \mathbb{T}, \mathbb{T}_0, \mathbb{O}, R, N)$
Input: vulnerability report R; target repository $\mathcal{D}$
Output: τ

Initialize world state: $s_0 \leftarrow (\mathcal{D}, \text{PoE}_0 = \emptyset)$
Initialize observation: $o_0 \leftarrow R$
Initialize trajectory: $\tau \leftarrow (o_0)$
Initialize policy $\pi(\tau)$:
for $t = 0, 1, 2, \ldots, N-1$ **do**
    Select action: $a_t \leftarrow \pi(\tau)$
    if $a_t =$ TERMINATE **then**
        break
    end if
    $s_{t+1} \sim \mathbb{T}(\cdot \mid s_t, a_t)$
    $o_{t+1} \sim \mathbb{O}(\cdot \mid s_{t+1})$
    $\tau \leftarrow (\tau, (a_t, o_{t+1}))$
end for
return τ

4.2 PoE Specification and Runner

To enable PoE-agent to construct and validate exploits, we introduce two core components: a *PoE specification* that formally represents an exploit, and a *PoE runner* that executes the specification and provides structured feedback.

The PoE specification serves as the intermediate representation used by the agent to incrementally construct and refine exploit attempts. The PoE runner interprets this specification and executes the corresponding transaction sequence, producing execution results that guide subsequent refinement.

PoE Specification Prior systems represent a PoE as executable source code [1]. While such representations are expressive, they introduce practical challenges that hinder

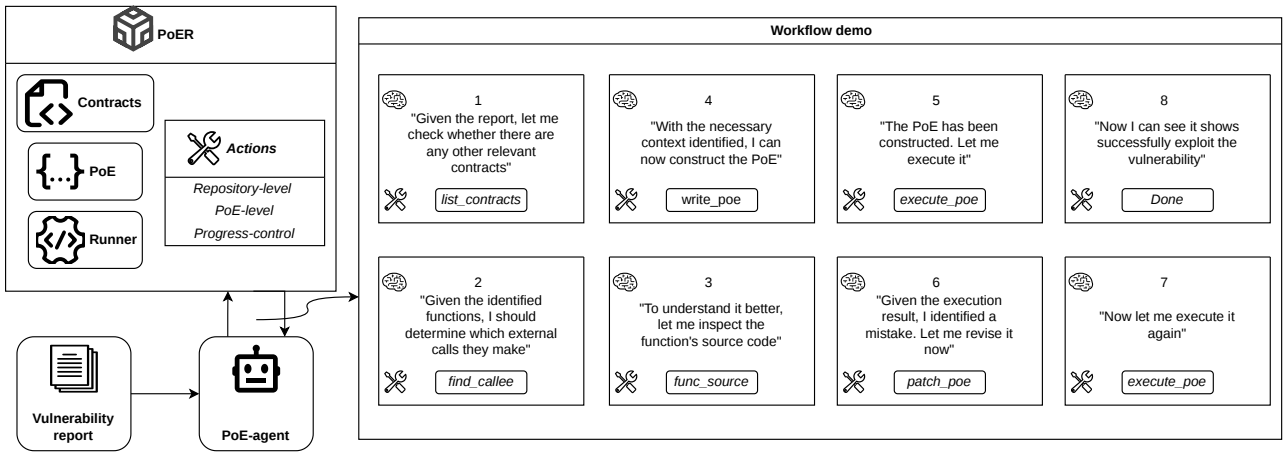


Figure 1: Workflow of PoE-agent. The PoE-agent agent iteratively reasons over the vulnerability report and target repository, constructs and edits an executable PoE, and validates it via execution feedback.

efficient and reliable PoE construction in an automated setting. In particular, treating PoEs as source code leads to the following limitations:

1. **Extraneous construction overhead.** Representing a PoE as executable source code entangles exploit logic with language- and framework-specific artifacts. Constructing or editing such PoEs often requires managing imports, resolving dependencies, handling language-specific syntax constraints, and generating auxiliary code or comments that are unrelated to the exploit itself. These incidental concerns unnecessarily bloat the PoE and substantially increase the cost of iterative refinement.

2. **Unsafe modification surface.** Treating the PoE as source code exposes the agent to a broad and unsafe editing surface. In particular, actions that modify executable code may inadvertently alter contracts in the target repository beyond the intended execution scenario, effectively changing the program under test rather than the exploitation environment. Such modifications undermine the validity and reproducibility of the resulting PoE.

3. **Uninformative compilation and deployment feedback.** Failures arising during compilation provide coarse and indirect signals to the agent. Error messages frequently conflate exploit logic errors with missing dependencies, incompatible compiler settings, or build-configuration issues, making it difficult to localize the root cause of a failed attempt and to perform targeted corrections.

Listing 5: An illustrative but incomplete PoE JSON specification for exploiting the "reentrancy" vulnerability.

```
{
  "actors": ["deployer", "victim", "attacker"],
  "action_sequence": [
    {
      "type": "deploy",
      "label": "Deploy SimpleVault",
      "contract_id": "vault",
      "artifact": "SimpleVault",
      "from": "deployer"
    },
    ...
    {
      "type": "call",
      "label": "Victim deposits 10 ETH into vault",
      "from": "victim",
      "to": "vault",
      "function": "deposit",
      "value": "10000000000000000000"
    },
    ...
    {
      "type": "data_op",
      "name": "attacker_balance_after",
      "description": "Attacker contract ETH
          balance after exploit",
      "data_type": "bigint",
      "immutable": false,
      "function": {
        "type": "contract_call",
        "contract": "attackerCon",
        "function": "getBalance",
      }
    },
    {
      "type": "assertion",
      "name": "attacker_profits",
      "kind": "security",
```

```
      "expression": {
        "op": ">",
        "lhs": "attacker_balance_after",
        "rhs": "attacker_balance_before"
      },
      "expected": true
    }
  ]
}
```

To mitigate above limitations, PoE-agent represents each PoE using a structured JSON specification rather than source code. Listing 5 shows an example. The JSON specification fully describes the PoE by listing external actors, an ordered PoE action sequence covering deployments, calls, EVM operations, state queries, data operations, and assertions, and the expressions used to bind or validate state. It supports both EOA-level transactions and message-level calls, allowing the modeling of malicious contracts required for advanced exploitation patterns such as reentrancy.

Using a structured specification offers several advantages. First, it allows the agent to reason about exploitation logic independently of language- or framework-specific syntax, avoiding the need to generate imports, boilerplate, or auxiliary scaffolding. Second, it prevents the agent from directly modifying source code in the target repository, ensuring that PoE construction is confined to the execution scenario rather than the program under test.

This abstraction is sufficiently expressive to represent multi-contract deployments, attacker-controlled interactions, and complex state transitions, while remaining deterministic, editable, and amenable to iterative refinement. Detailed schema definitions appear in the appendix A.

PoE Runner The PoE runner bridges specification and execution. It interprets the JSON PoE specification and executes the described transaction sequence in a deterministic EVM environment, producing structured execution feedback that guides subsequent refinement.

Execution Result Information When a transaction sequence is executed, the PoE runner provides structured feedback regarding the outcome. For failed executions, the environment reports the root cause through a single transaction object representing the top-level call submitted to the blockchain. This object captures the essential context of the failure: the sender address, the target contract address, and the encoded function call data. A descriptive reason is also provided to explain the nature of the revert (e.g., an assertion failure or an access control violation).

While the reported transaction only represents the outermost call, internal contract-to-contract interactions—such as token transfers or re-entrant calls—are often the actual site of the vulnerability. These internal interactions are not implicitly included in the standard error response but can be recovered through explicit execution tracing. The agent leverages the top-level call context and the failure reason to diagnose the execution state, inferring whether the failure stems from malformed inputs, incorrect environmental setup, or a successfully triggered security violation.

Formally, we define the execution result as a tuple that distinguishes between successful and failed outcomes:

$$\text{ExecResult} ::= \begin{cases} \text{Success}(\sigma_{t+1}) \\ \text{Failure}(\mathit{reason}, \mathit{tx}, \mathit{trace}^*) \end{cases} \tag{4}$$

where σ_{t+1} represents the updated world state after a successful transition; *reason* denotes the failure explanation; tx is the top-level transaction context $\langle from, to, data \rangle$; and $trace^*$ is an optional sequence of internal call frames. In the case of success, the result primarily serves to confirm the new state, whereas in the case of failure, the additional context is used by the agent to iteratively refine the exploit sequence.

4.3 Action Space Design

To enable PoE-agent to operate under incomplete information and iteratively refine a PoE, we design a structured action space that exposes repository semantics and PoE modification capabilities in a controlled manner.

The action space $\mathcal{A}$ is organized into three functional categories: *repository-level actions*, *PoE-level actions*, and *progress-control actions*. This separation reflects three complementary needs in PoE construction: (1) acquiring semantic knowledge from the target repository, (2) constructing and iteratively editing the PoE specification, and (3) explicitly tracking progress and determining when execution should terminate. The full details are in appendix B.

4.3.1 Repository-Level Actions

Repository-level actions provide the agent with a structured interface for inspecting the target codebase. The design of these actions follows two guiding principles: *effectiveness* and *efficiency*.

Effectiveness requires that the action space be expressive enough to expose all information necessary for PoE construction. For example, this can simply include access to the complete source code of relevant contracts, allowing the agent to reason about precise control flow, state updates, and security checks.

However, naively exposing full source files for every reasoning step is inefficient and unnecessary.Large repositories introduce substantial context overhead, which can impair both reasoning precision and execution speed.

Static Analysis Integration. To improve efficiency, PoE-agent incorporates repository-level actions backed by a static analysis tool, Slither [6].

These actions directly surface high-level semantic information, such as function signatures, state variables, call relationships, and guard conditions, without requiring the agent to process large volumes of raw code.

For example, actions that resolve caller–callee relationships or identify relevant dependency contracts allow the agent to efficiently reconstruct cross-contract execution paths that are critical for exploit construction. By combining selective source inspection with targeted static analysis, PoE-agent ensures that the agent can obtain complete execution-relevant information while minimizing redundant analysis and interaction overhead.

Concretely, repository-level actions enable the agent to: (i) enumerate contracts and their externally callable functions; (ii) inspect function implementations and state variables; and (iii) resolve cross-contract call relationships. Such actions are essential for identifying vulnerable entry points, implicit dependencies, and auxiliary contracts that are not explicitly mentioned in the vulnerability report.

By exposing repository structure through read-only, analyzable actions, PoE-agent allows the agent to infer missing components while remaining grounded in the actual codebase.

4.3.2 PoE-Level Actions

PoE-level actions operate directly on the structured PoE specification under construction. Instead of generating raw Solidity or test code, the agent manipulates a JSON-based representation of the PoE that explicitly encodes the precondition, the ordered transaction sequence $\vec{T}$, and the postcondition.

These actions allow the agent to: (i) initialize a PoE specification from the information available in the vulnerability report; (ii) inspect or retrieve specific fields of the current PoE specification and (iii) incrementally edit the PoE JSON file (e.g. transaction sequence, arguments, etc.) based on the execution feedback. The full details can be found in appendix A.3

4.3.3 Progress-control Actions

PoE-agent includes dedicated actions whose sole purpose is to track internal progress and to explicitly terminate execution with one of two final outcomes: *complete* or *out-of-scope*. Explicit progress tracking is essential for managing long-horizon, context-intensive reasoning while preserving disciplined and deterministic control flow.

To support this, PoE-agent maintains four boolean conditions: INFOREADY, POEEXISTS, POEPASSES, and OUTOFSCOPE. INFOREADY holds when the agent has identified all required external actors EA, the set of contracts C to be deployed, and the function signatures F that together define a complete transaction sequence $\vec{T}$. POEEXISTS indicates that a concrete PoE JSON specification has been constructed. POEPASSES holds when executing the PoE reaches a final state that violates the reported security property *Prop*. OUTOFSCOPE holds when tool evidence indicates that the PoE cannot be instantiated under the given repository without modifying source files or build configuration.

These conditions serve two complementary roles. First, they provide a precise, explicit notion of progress during PoE construction, distinguishing between missing semantic information, incomplete specifications, and execution-level outcomes. Second, they define clear termination criteria. When either POEPASSES or OUTOFSCOPE becomes true, the agent emits a dedicated reporting action and halts further execution. This explicit termination mechanism ensures that every PoE-agent run concludes in a well-defined and auditable state.

Out-of-Scope Executions Some failures indicate that a PoE cannot be instantiated under the given repository without modifying source code or build configuration; we classify such cases as *out-of-scope*. Representative examples include compilation failures caused by missing dependencies, references to nonexistent interfaces, or protocol-level deployment violations such as exceeding the maximum contract size limit. Because these failures cannot be resolved through transaction-level refinement alone, PoE-agent terminates execution immediately upon detecting them.

5 Evaluation

Research Questions. This section evaluates POE-AGENT with respect to the following research questions:

- **R1 (Effectiveness).** To what extent can PoE be automatically constructed from vulnerability reports?
- **R2 (Failure Analysis).** What failure modes and error types occur during automated PoE generation, and what do they reveal about the limitations of current analysis and reasoning capabilities?
- **R3 (Efficiency).** To what extent does automated PoE generation reduce the cost required for vulnerability validation compared to traditional, human-driven workflows?

5.1 Experiment Setup

Dataset We evaluate POE-AGENT on **PoE-bench**, a curated dataset of real-world smart contract vulnerabilities derived from publicly disclosed audit reports [26]. Each instance consists of a vulnerability report paired with the corresponding audited contract repository.

To ensure executable evaluation and practical relevance, we select vulnerabilities according to the following criteria: (i) the target repository is publicly available and can be successfully compiled in a reproducible execution environment; and (ii) the dataset spans multiple vulnerability categories and independent projects to preserve structural and semantic diversity.

The resulting PoE-bench dataset contains 56 exploitable vulnerabilities across 18 repositories. For each repository, we measure source lines of code (SLOC) using `cloc` [4] and analyze contract structure with Slither [6], counting concrete contract definitions and implemented functions. Table 2 summarizes repository-level characteristics, and Table 3 presents the distribution of vulnerability types.

Language Model Configuration. PoE-agent is instantiated with a single reasoning-capable LLM, DeepSeek v3.2. To ensure experimental consistency and eliminate confounding factors, the same model version and prompt configuration are used across all evaluations. For reproducibility, the model temperature is set to 0.

Table 2: Project Characteristics in the Evaluation Dataset

Metric	Min	Max	Mean	Median
#Contracts	6	207	69.44	62.5
#Functions	49	2583	855.94	827.5
SLOC	378	13209	3834.40	2988.0

PoE-agent Sandbox Environment To execute and validate generated PoEs, we implement a runtime execution environment sandbox, *POER*, that packages the target contract repository together with the PoE specification and the PoE runner. The environment provides a deterministic EVM-based execution context in which the runner interprets the JSON PoE specification and executes the corresponding transaction sequence.

For each vulnerability instance, the environment initializes the contract repository, installs required dependencies, and compiles the contracts using the configuration specified in the repository. The PoE runner then deploys contracts and executes the ordered action sequence defined in the PoE specification. Execution outcomes, including transaction results, revert reasons, and optional execution traces, are returned as structured feedback to guide iterative refinement by the agent.

This environment isolates the exploit execution scenario from the original source code, ensuring that PoEs are evaluated without modifying the target repository and that experiments remain reproducible across different vulnerability instances.

5.2 R1: Effectiveness

Effectiveness Metric. To answer R1, we define a PoE as successful if it satisfies all three of the following criteria:

1. **Executability**: the PoE executes without errors in the evaluation environment;
2. **Security Violation**: executing the PoE transitions the system from the initial state σ_t to a state σ_{t+1} that violates the intended security property described in the vulnerability report;
3. **Threat-Model Consistency**: the exploit relies only on realistic contract interactions and assumptions consistent with the reported threat model.

Table 3: Distribution of vulnerability types in the dataset. Vulnerability categories follow the L- and S-class taxonomy (https://github.com/ZhangZhuoSJTU/Web3Bugs/blob/main/docs/standard.md).

Vulnerability Type	Count
Reentrancy (L1)	2
Rounding / Precision Errors (L2)	2
Uninitialized Variables (L3)	2
Gas-Related Errors (L4)	1
Storage / Proxy Confusion (L5)	3
Integer Overflow / Underflow (L7)	1
Low-level Call / Revert Errors (L8)	3
Cryptographic Issues (LA)	3
`tx.origin` Misuse (LB)	2
Oracle Manipulation (S1)	7
ID-Related Violations (S2)	3
Erroneous State Updates (S3)	1
Business-Flow Atomicity Violations (S4)	3
Access Control Violations (S5)	5
Erroneous Accounting (S6)	9
Broken Business Logic (SE)	6
Contract-Specific Bugs (SC)	3
Total	**56**

Overall Effectiveness. PoE-agent generates executable PoEs for a substantial portion of vulnerabilities in **PoE-bench**. Excluding instances that are correctly identified as true out-of-scope (TO), we define the success rate as:

$$\text{Success Rate} = \frac{\#\text{Success}}{\#\text{Total} - \#\text{TO}}.$$

Across all evaluated vulnerabilities, PoE-agent achieves an overall success rate of 43.6% (24/55).

Table 4 summarizes outcomes by vulnerability category.

5.3 R2: Failure Analysis

Failure Classification. To answer R2, we analyze unsuccessful runs based on the *final state of the agent*. Although the framework exposes only two explicit final statuses (`completed` and `stop`), manual inspection reveals three recurring failure modes that correspond to distinct error types in automated PoE construction.

- **False Completion (FC).** The agent reports `completed` even though the generated PoE is not valid under our definition.

Type	Total	Success	FO	TO	NA	FC
L1	2	0	2	0	0	0
L2	2	0	1	0	1	0
L3	2	1	0	0	1	0
L4	1	1	0	0	0	0
L5	3	1	1	0	1	0
L7	1	0	1	0	0	0
L8	3	1	2	0	0	0
LA	3	2	1	0	0	0
LB	2	0	2	0	0	0
S1	7	5	2	0	0	0
S2	3	2	1	0	0	0
S3	1	0	1	0	0	0
S4	3	2	0	0	0	1
S5	5	3	2	0	0	0
S6	9	3	5	1	0	0
SC	3	0	1	0	0	1
SE	6	3	3	0	0	0
Total	**56**	**24**	**25**	**1**	**3**	**2**

Table 4: Outcome breakdown by vulnerability category. FO: False Out-of-Scope; TO: True Out-of-Scope; NA: No Action; FC: False Completion.

- **False Out-of-Scope (FO).** The agent terminates with `stop` after incorrectly concluding that exploitation is out of scope, despite the existence of a feasible PoE.
- **No Action (NA).** The agent terminates without report'complete' or 'out of scope'.

Table 4 shows failure outcomes by vulnerability category.

5.4 R3: Efficiency

Token Usage and Cost. To quantify the computational cost of automated PoE generation, we record the total number of input and output tokens consumed per instance, as well as the corresponding monetary cost. Table 5 summarizes these statistics over all evaluated instances and over successfully completed PoE generations.

The maximum, mean, and median token usage across *all* cases are consistently higher than the corresponding statistics among *successful* cases. This gap suggests that unsuccessful instances are generally harder: they require broader exploration or larger context to attempt resolution, which drives additional token consumption. Thus, failures are not simply due to insufficient computation, but to more complex reasoning and contextual challenges.

6 Threats to Validity

In this section, we discuss the potential threats to the validity of our evaluation and the steps we have taken to mitigate them.

Table 5: Token usage and cost statistics for automated PoE generation

Subset	Metric	Min	Max	Mean	Median
All (56)	Input tokens	170,508	4,127,425	1,311,184	1,126,314
	Output tokens	1,744	24,051	11,811	11,719
	Cost (USD)	0.00812	0.13829	0.04742	0.04212
Success (24)	Input tokens	179,495	2,443,140	1,264,567	1,151,827
	Output tokens	1,744	22,146	11,687	11,795
	Cost (USD)	0.00812	0.08390	0.04583	0.04237

Internal validity

- **LLM Non-determinism**: The performance of our framework relies on LLMs, which are inherently non-deterministic. Different runs with the same input may yield different PoEs. To mitigate this, we set the temperature to 0 to minimize variance.
- **Context Engineering**: The quality of the generated PoEs is sensitive to the design of the prompts and the selection of tools provided to the agent. We followed a systematic approach to prompt design, but there may still be room for optimization that could affect the results.
- **Human Subjectivity in Evaluation**: The manual review of PoEs by security experts involves subjective judgment and errors. To minimize this, we established clear criteria for success.

External validity

- **Dataset Representativeness**: Our evaluation is based on vulnerabilities from Code4rena. While Code4rena covers a wide range of real-world projects, it may not represent all possible smart contract architectures or vulnerability types found in other platforms or private audits.
- **Language and Framework Specificity**: Our current implementation and evaluation focus on Solidity and the EVM ecosystem. While the core concepts of our framework are language-agnostic, the specific tools and the JSON-to-PoE runner are tailored to this environment. The results might differ for other smart contract languages (e.g., Rust for Solana or Move).
- **LLM Model**: Our experiments were conducted using Deepseek-V3.2. The rapid evolution of LLMs means that newer models might perform significantly better or differently, potentially changing the effectiveness and efficiency metrics of our framework.

7 Related Work

Automatically generating exploits or PoC artifacts for smart contract vulnerabilities has been studied from multiple perspectives, including fuzzing-based testing, program analysis and symbolic execution, and more recently LLM based approaches.

Fuzzing-Based Exploit Generation. Early research explored the use of fuzzing techniques to automatically discover and reproduce vulnerabilities in smart contracts. Tools such as ContractFuzzer [10] performs ABI-guided input generation and instruments the Ethereum Virtual Machine (EVM) to monitor runtime behaviors. By analyzing execution logs against predefined vulnerability oracles, it detects security violations. Subsequent work improved fuzzing efficiency and coverage by incorporating adpative strategies. For example, sFuzz [17] applies adaptive fuzzing strategies tailored for Solidity contracts, while ILF [9] leverages imitation learning to guide fuzzing toward security-relevant execution paths. Similarly, Echidna [8] performs property-based fuzz testing to detect violations of user-defined invariants. More recently, VERITE [11] introduces a profit-oriented fuzzing framework designed for DeFi vulnerabilities. It incorporates action-based transaction mutators to explore diverse fund-flow patterns and employs gradient-based optimization to maximize the profitability of discovered exploits, enabling automated extraction of concrete attack transactions from vulnerable contracts. Although these tools are effective for vulnerability discovery, their outputs typically consist of failing test cases or execution traces rather than structured PoC exploits suitable for audit reports. Consequently, additional manual effort is often required to transform fuzzing outputs into reproducible exploit demonstrations.

Program Analysis and Symbolic Execution. Another line of work applies static analysis and symbolic execution to derive exploit conditions and generate concrete attack transactions. Tools such as Oyente [13] and Mythril [5] analyze Ethereum bytecode to detect potential vulnerabilities using symbolic reasoning over execution paths. More recent systems attempt to construct concrete exploits from these analysis results. For instance, symbolic execution engines can synthesize transaction inputs that satisfy vulnerability-triggering constraints, enabling automatic reproduction of attacks. Similarly, Manticore [15] allow the generation of concrete inputs that lead to security violations. While these techniques provide strong reasoning capabilities over program semantics, they often struggle with path explosion and environment modeling in complex decentralized finance (DeFi) systems. Moreover, the generated exploits are usually low-level transaction sequences rather than human-readable PoC artifacts that can be directly integrated into audit workflows.

Agentic Exploit Generation. Recent work has explored the use of agentic LLM systems to automate complex software engineering and security tasks. Agentic frameworks enable LLMs to iteratively reason, invoke tools, and refine outputs through interaction with external environments. Systems such as ReAct [25] and tool-augmented LLM frameworks demonstrate that combining reasoning with tool execution allows models to solve multi-step tasks that require information retrieval, program analysis, and environment feedback. These approaches have inspired the development of autonomous agents for software engineering tasks such as program repair, vulnerability analysis, and exploit construction.

Recently, Prompt-to-Pwn [21] demonstrates that LLMs can translate vulnerability reports into executable exploit scripts. Similarly, SmartPoC [3] generates validated PoCs from bug reports by iteratively refining code using execution feedback. PoCo [1] proposes an agentic framework in which an LLM interacts with development tools such as compilers and test frameworks in a reasoning–action loop to synthesize PoC exploits. Other approaches explore combining LLMs with formal verification or historical exploit data.

These approaches demonstrate the promise of LLMs in automating exploit construction, but they often focus on individual contracts or limited contexts, which restricts their ability to reason about repository-wide dependencies and complex cross-contract interactions.

In contrast, POE-AGENT focuses on generating structured Pos, enabling more reliable reasoning about repository-wide interactions and exploit construction.

8 Conclusion

This paper presented POE-AGENT, an agentic framework that automatically generates executable PoEs from vulnerability reports and smart contract repositories. By integrating targeted program analysis with iterative LLM reasoning and execution feedback, POE-AGENT bridges the gap between informal reports and concrete, machine-checkable exploit artifacts.

We evaluated POE-AGENT on PoE-bench, a benchmark of 56 real-world vulnerabilities spanning 17 categories, and achieved a 43.6% success rate (24/55, excluding true out-of-scope cases). Our failure analysis isolates distinct error modes, underscoring that the remaining gaps stem primarily from contextual reconstruction and termination judgment rather than execution capability. We also show that automated PoE construction is computationally efficient and produces reusable exploit skeletons even in unsuccessful cases, reducing manual validation effort.

Overall, our results indicate that agentic LLM systems can operationalize vulnerability validation for smart contracts at repository scale. Future work will focus on improving context recovery for underspecified reports, strengthening stopping criteria to reduce false completions, and extending the framework to cross-contract and economically driven exploit scenarios.

References

[1] Vivi Andersson, Sofia Bobadilla, Harald Hobbelhagen, and Martin Monperrus. PoCo: Agentic proof-of-concept exploit generation for smart contracts. arXiv, November 2025. Available: https://arxiv.org/abs/2511.02780, accessed February 28, 2026.

[2] Vini Barbosa. Ethereum deploys record 8.7M contracts in Q4 2025: What's next for ETH? Yahoo Finance, December 2025. Available: https://finance.yahoo.com/news/ethereum-deploys-record-8-7m-145351312.html, accessed February 28, 2026.

[3] Longfei Chen, Ruibin Yan, Taiyu Wong, Yiyang Chen, and Chao Zhang. SmartPoC: Generating Executable and Validated PoCs for Smart Contract Bug Reports. arXiv, November 2025. Available: https://arxiv.org/abs/2511.12993, accessed February 28, 2026.

[4] Albert Danial. cloc: v1.92, December 2021.

[5] ConsenSys Diligence. Mythril: Security analysis tool for ethereum smart contracts. https://github.com/Consensys/mythril. accessed Februrary 28, 2026.

[6] Josselin Feist, Gustavo Grieco, and Alex Groce. Slither: a static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd international workshop on emerging trends in software engineering for blockchain (WETSEB)*, pages 8–15. IEEE, 2019.

[7] Arthur Gervais and Liyi Zhou. AI Agent Smart Contract Exploit Generation. arXiv, January 2026. Available: https://arxiv.org/abs/2507.05558, accessed February 28, 2026.

[8] Gustavo Grieco, Will Song, Artur Cygan, Josselin Feist, and Alex Groce. Echidna: Effective, usable, and fast fuzzing for smart contracts. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 557–560. ACM, July 2020.

[9] Jingxuan He, Mislav Balunović, Nodar Ambroladze, Petar Tsankov, and Martin Vechev. Learning to Fuzz from Symbolic Execution with Application to Smart Contracts. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 531–548. ACM, November 2019.

[10] Bo Jiang, Ye Liu, and W. K. Chan. ContractFuzzer: Fuzzing smart contracts for vulnerability detection. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, pages 259–269. ACM, September 2018.

[11] Ziqiao Kong, Cen Zhang, Maoyi Xie, Ming Hu, Yue Xue, Ye Liu, Haijun Wang, and Yang Liu. Smart contract fuzzing towards profitable vulnerabilities. *Proceedings of the ACM on Software Engineering*, 2:153–175, June 2025.

[12] Matthew Kosinski. What is the Vulnerability Management Lifecycle? IBM Think, July 2023. Available: https://www.ibm.com/think/topics/vulnerability-management-lifecycle, accessed February 28, 2026.

[13] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making Smart Contracts Smarter. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 254–269. ACM, October 2016.

[14] Oonagh McDonald. *Appendix: smart contracts*, page 225–228. Agenda Publishing, 2023.

[15] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts.

[16] Andrew Ng. Welcoming Diverse Approaches Keeps Machine Learning Strong, June 2024.

[17] Tai D. Nguyen, Long H. Pham, Jun Sun, Yun Lin, and Quang Tran Minh. sFuzz: An Efficient Adaptive Fuzzer for Solidity Smart Contracts.

[18] Murugiah Souppaya and Karen Scarfone. Guide to Enterprise Patch Management Planning: Preventive Maintenance for Technology. Technical report, National Institute of Standards and Technology, November 2021.

[19] Ángel Jesús Varela-Vaca and Antonia M Reina Quintero. Smart contract languages: A multivocal mapping study. *ACM Computing Surveys (CSUR)*, 54(1):1–38, 2021.

[20] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper, 2014. Available: https://ethereum.github.io/yellowpaper/paper.pdf, accessed February 28, 2026.

[21] Zeke Xiao, Qin Wang, Yuekang Li, and Shiping Chen. Prompt to Pwn: Automated Exploit Generation for Smart Contracts. arXiv, January 2026. Available: https://arxiv.org/abs/2508.01371, accessed February 28, 2026.

[22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.

[23] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. $τ$-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains, June 2024.

[24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

[26] Zhuo Zhang, Brian Zhang, Wen Xu, and Zhiqiang Lin. Demystifying Exploitable Bugs in Smart Contracts. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 615–627, Melbourne, Australia, May 2023. IEEE.

A PoE Specification and Execution

This appendix formally specifies the JSON schema used to represent a Proof of Exploitation (PoE). The schema defines the structure of actors, executable actions, data definitions, and property assertions required to reproduce a vulnerability scenario in a deterministic execution environment. An example PoE is given in Listing 6.

A.1 Root Object

A PoE instance is a JSON object with the following top-level fields:

- **`actors`**: Array of external actor identifiers.
- **`action_sequence`**: Ordered array of action objects (see Section A.3).

A.2 Actors

Actors represent distinct externally owned accounts (EOAs) participating in the scenario. The `actors` field is an array of strings. Signer indices are assigned deterministically according to array position (index 0, 1, 2, . . .).

Type: `string[]`

A.3 Action Sequence and Types

The `action_sequence` field is a flat array of action objects executed strictly in the order in which they appear in the array. Execution order is determined solely by array position; no explicit ordering field is required.

Each action object must contain the following base field:

- **`type`** (string): Action type.

The following optional base field may be included:

- **`label`** (string): Human-readable description for debugging or documentation purposes.

The schema supports the following action types:

1. **`deploy`** — Deploys a contract instance and registers it under a `contract_id` for subsequent reference.
2. **`call`** — Executes a function call on a deployed contract, optionally transferring value and optionally binding returned or emitted addresses.
3. **`evm_increaseTime`** — Advances the block timestamp by a specified number of seconds.
4. **`evm_mine`** — Mines one or more blocks.
5. **`evm_setCode`** — Sets runtime bytecode at a specified address using the artifact's compiled runtime code. This enables modeling hardcoded or pre-deployed contracts (e.g., canonical token addresses).
6. **`state_query`** — Executes a view or pure function and stores the result under a symbolic name.
7. **`data_op`** — Defines a computed or constant symbolic value that can be referenced in subsequent expressions.
8. **`assertion`** — Evaluates a logical expression and verifies whether the expected property holds.
9. **`contract_mock`** — Introduces an auxiliary or malicious contract not present in the original repository, enabling modeling of external dependencies or adversarial behavior.

Collectively, these action types provide complete coverage for modeling multi-step, multi-actor exploit scenarios, including state manipulation, mocking of dependencies, and deterministic verification of security properties.

A.4 Deploy Action

`type = "deploy"`

- **`contract_id`** (string): Identifier for later reference.
- **`artifact`** (string): Artifact path or contract name.
- **`from`** (string): Deploying actor.
- **`args`** (optional array): Constructor arguments.
- **`value`** (optional string): Ether value (wei).
- **`libraries`** (optional object): Library linking map.

Contracts must be deployed before being referenced in subsequent actions.

A.5 Call Action

`type = "call"`

- **`actor`** (string): Calling actor.
- **`contract`** (string): Target contract identifier.
- **`function`** (string): Function name or signature.
- **`args`** (optional array): Function arguments.
- **`value`** (optional string): Ether value.
- **`expect_revert`** (optional boolean): Whether revert is expected.
- **`bind`** (optional object): Binding configuration for returned or emitted addresses.

A.6 Binding Semantics

The `bind` object captures dynamically created contract addresses for later use.

- **`contract_id`**: Identifier for the new contract.
- **`artifact`** (optional): ABI reference.
- **`source`**: One of `"return"`, `"event"`, `"create2"`.

Event-based and CREATE2-based extraction require additional configuration parameters.

A.7 State Query Action

`type = "state_query"`

- **`name`**: Identifier for stored result.
- **`contract`**: Target contract.
- **`function`**: View/pure function.
- **`args`** (optional): Arguments.

A.8 Data Operation Action

`type = "data_op"`

- **`name`**: Data identifier.
- **`data_type`**: `"bigint"`, `"number"`, `"string"`, or `"boolean"`.
- **`immutable`**: Boolean.
- **`value`** (optional): Constant value.
- **`function`** (optional): Expression.

Immutable entries with constant dependencies are evaluated eagerly; others are evaluated lazily. Snapshot semantics are supported via the `@name` reference syntax.

A.9 Assertion Action

`type = "assertion"`

- **`name`**: Assertion identifier.
- **`expression`**: Expression tree.
- **`expected`**: Boolean indicating whether the property should hold.

Execution halts if an assertion fails.

A.10 Expressions

Expressions are used in `data_op` and `assertion` actions.

An expression (`Expr`) may be:

- **String** — Reference to a state query name, data entry name, snapshot reference (using `@name`), or bigint literal.
- **Number** — Bigint literal.
- **Binary operation object**:
  - `op`: arithmetic or comparison operator
  - `lhs`: left expression
  - `rhs`: right expression

  Supported operators include: `+`, `-`, `*`, `/`, `==`, `!=`, `<`, `>`, `<=`, `>=`.
- **Contract call expression**:
  - `type = "contract_call"`
  - `contract`: contract identifier
  - `function`: view function name
  - `args` (optional): arguments
- **Ternary expression**:

  ```
  condition ? thenValue : elseValue
  ```

  encoded as:

  ```
  { "?": [condition, thenValue, elseValue] }
  ```

A.11 Execution Semantics

Actions execute sequentially according to ascending `idx`. During execution:

1. Transactions modify blockchain state.
2. State queries record intermediate state.
3. Data operations define reusable computed values.
4. Assertions evaluate security properties.

Execution terminates upon assertion failure or unrecoverable action error.

A.12 Mocking

The `contract_mock` action enables definition of auxiliary contracts not present in the original repository. This mechanism supports modeling malicious helper contracts and standard external dependencies (e.g., ERC20, ERC721).

A.13 Design Principles

The schema is designed to:

- Separate exploit logic from framework-specific test code,
- Provide deterministic execution ordering,
- Support multi-actor, multi-step exploit scenarios,
- Enable formal property verification.

Only fields relevant to a given action type should be included.

Listing 6: PoE JSON specification for a reentrancy exploit.

```
{
  "actors": ["deployer", "victim", "attacker"],
  "action_sequence": [
    {
      "type": "deploy",
      "label": "Deploy SimpleVault",
      "contract_id": "vault",
      "artifact": "SimpleVault",
      "from": "deployer"
    },
    {
      "type": "deploy",
      "label": "Deploy ReenterExploit",
      "contract_id": "attackerCon",
      "artifact": "ReenterExploit",
      "from": "attacker",
      "args": ["vault"]
    },
    {
      "type": "call",
      "label": "Victim deposits 10 ETH into vault",
      "from": "victim",
      "to": "vault",
      "function": "deposit",
      "value": "10000000000000000000"
    },
    {
      "type": "data_op",
      "name": "attacker_balance_before",
      "label": "Attacker contract ETH balance
          before exploit",
      "data_type": "bigint",
      "immutable": false,
      "function": {
        "type": "contract_call",
        "contract": "attackerCon",
        "function": "getBalance"
      }
    },
    {
      "type": "call",
      "label": "Seed attacker contract with 1 ETH",
      "from": "attacker",
      "to": "attackerCon",
      "function": "seed",
      "value": "1000000000000000000"
    },
    {
      "type": "call",
      "label": "Execute reentrancy attack",
      "from": "attacker",
      "to": "attackerCon",
      "function": "attack",
    },
    {
      "type": "data_op",
      "name": "attacker_balance_after",
      "description": "Attacker contract ETH
          balance after exploit",
      "data_type": "bigint",
      "immutable": false,
      "function": {
        "type": "contract_call",
        "contract": "attackerCon",
        "function": "getBalance",
      }
    },
    {
      "type": "assertion",
      "name": "attacker_profits",
      "kind": "security",
      "expression": {
        "op": ">",
        "lhs": "attacker_balance_after",
        "rhs": "attacker_balance_before"
      },
      "expected": true
    }
  ]
}
```

B Action Space Design

his appendix provides a complete specification of the action space $\mathcal{A}$ used by PoE-agent, organized into the three functional categories introduced in Section 4.3.

B.1 Repository-Level Actions

Repository-level actions enable the agent to inspect the target codebase through both direct file access and static analysis queries. These actions are read-only and do not modify the repository state.

Contract Enumeration

- `list_all_contracts(include_non_derived)`: Returns a list of all contracts in the repository. By default, returns only derived contracts (leaf nodes in the inheritance graph); setting `include_non_derived=True` includes base contracts and interfaces.

- `get_contract_file_path(contract_name)`: Returns the absolute file path where a contract is defined.

Function Inspection

- `get_functions(contract_name, declarer)`: Returns all public/external function signatures for a contract. Filters to only implemented functions (excludes interface declarations). If `declarer` is provided, returns only functions declared in that specific contract; otherwise returns all functions available in the contract (including inherited).
- `get_function_source(contract_name, function)`: Retrieves the Solidity source code for a function. Accepts either a full function signature (e.g., `"transfer(address,uint256)"`) or a function name (e.g., `"transfer"`). If multiple overloaded functions exist, returns all matching implementations.
- `get_func_guards(contract_name, function_sig, max_guard_len)`: Extracts all `require` and `revert` guard conditions for a function. Guards longer than `max_guard_len` (default 160 characters) are truncated.
- `get_func_modifiers(contract_name, function_sig)`: Returns the list of modifiers applied to a function.
- `get_func_rw(contract_name, function_sig)`: Returns state variables read and written by a function during execution.

State and Event Inspection

- `get_state_vars(contract_name)`: Returns all state variables defined in a contract, including their types and visibility.
- `get_events(contract_name)`: Returns all events declared in a contract.

Cross-Contract Analysis

- `find_callers(contract_name, function_signature)`: Performs static analysis to identify all functions that invoke the specified contract function. Accepts a single function signature or a list of signatures. Supports both full signatures and bare function names; bare names are expanded to all matching overloaded implementations.
- `find_callees(contract_name, function_signature)`: Identifies all functions invoked by the specified contract function, including both direct calls and calls through interfaces. Supports the same signature formats as `find_callers`.

File Access

- `view_file(file_path, start_line, end_line)`: Returns the contents of a source file with line numbers. If `start_line` and `end_line` are provided, returns only the specified line range; otherwise returns the entire file.

All repository-level actions validate that contract data is loaded before execution and return structured error messages if contracts or functions are not found.

B.2 PoE-Level Actions

PoE-level actions operate on the structured JSON specification of the PoE under construction. The PoE JSON encodes the precondition, an ordered transaction sequence $\vec{T}$, and the postcondition.

PoE Initialization and Inspection

- `write_poe(poe)`: Initializes the PoE specification by storing a complete JSON document in agent state. The `poe` must be a valid dictionary containing required fields such as `action_sequence`, `actors`, etc.
- `get_poe(path)`: Retrieves specific components of the current PoE specification using path-based addressing. Supports:
  - `/action_sequence`: Returns all actions in the transaction sequence
  - `/action_sequence/`i: Returns the action at index i
  - `/action_sequence/(`i`:`j`)`: Returns actions from index i to j (inclusive)
  - `/action_sequence/(`i`,`j`)`: Returns actions at specific indices i and j

  All returned actions are annotated with their `idx` values for reference during editing.

PoE Editing

- `patch_poe(patch_ops)`: Applies a list of anchor-based JSON Patch operations to incrementally modify the PoE specification. The patch format extends RFC 6902 with anchor-based array addressing to handle index shifts during sequential edits. Supported operations:
  - `add`: Insert elements using anchor-based paths of the form `/action_sequence/`$k.m$, where k is an anchor index and m is an offset. Positive offsets ($k.1, k.2, \ldots$) insert after the anchor; negative offsets ($k.-1, k.-2, \ldots$) insert before.

- `remove`: Delete elements by anchor reference (e.g., `/action_sequence/`k).
- `replace`: Update existing field values (e.g., `/action_sequence/`k`/function`).
- `move`/`copy`: Move or copy elements using anchor references.

Anchor-based addressing ensures deterministic patch application even when multiple operations interact, as all positions are defined relative to a stable snapshot of the array rather than mutable indices.

PoE Execution

- `execute_poe()`: Executes the current PoE specification in a deterministic EVM environment and returns test results. Before execution, automatically validates library dependencies and applies auto-fix patches if missing libraries are detected. Returns:
  - Execution status and test pass/fail statistics
  - Detailed error information (reasons and traces) if tests fail
  - Auto-fix messages if library dependencies were automatically resolved

  Execution failures are classified as either fixable (requiring PoE refinement) or out-of-scope (indicating the PoE cannot be instantiated without source code modifications).

B.3 Progress and Termination Actions

Progress Tracking

- `update_progress(conditions)`: Explicitly records and updates the truth values of the four progress conditions (INFOREADY, POEEXISTS, POEPASSES, OUTOFSCOPE). The `conditions` parameter is a list of entries, where each entry specifies:
  - `proposition`: One of `"InfoReady"`, `"PoEExists"`, `"PoEPasses"`, or `"OutOfScope"`
  - `status`: One of `"unknown"` (not yet evaluated), `"false"` (evaluated and currently false), or `"true"` (evaluated and currently true)

  This action enables the agent to explicitly maintain its progress state during long-horizon reasoning. The progress state is stored within the agent state and can be updated incrementally as new evidence becomes available.

Termination Reporting

- `report(status)`: Explicitly terminates agent execution with a final status. The `status` parameter must be either `"completed"` (indicating successful PoE construction and execution) or `"stop"` (indicating the task is out-of-scope). This action transitions the agent to the terminal state and halts further execution.

The four progress conditions (INFOREADY, POEEXISTS, POEPASSES, OUTOFSCOPE) can be tracked either explicitly through `update_progress` or evaluated implicitly based on agent state and execution results. POEPASSES becomes true when `execute_poe` returns test results indicating all tests passed. OUTOFSCOPE becomes true when execution errors indicate the PoE cannot be instantiated without modifying source files or build configuration (e.g., compilation failures, missing dependencies, contract size violations).

B.4 Anchor-Based JSON Patch Details

The anchor-based patch system addresses ambiguity in sequential array edits by using stable anchor references instead of mutable array indices. Before applying any patch operations, the system records a snapshot $A^{(0)} = [a_0, a_1, \ldots, a_{n-1}]$ of each array. Each element $a_k^{(0)}$ serves as a stable anchor, uniquely identifiable via an immutable identifier field (typically `idx` for `action_sequence` arrays).

Patch operations refer to positions using extended identifiers of the form $k.m$, where k denotes the anchor index in $A^{(0)}$ and $m \in \mathbb{Z} \setminus \{0\}$ is a relative offset:

- Positive offsets ($k.1$, $k.2$, $\ldots$) denote positions immediately following anchor $a_k^{(0)}$, ordered by increasing m
- Negative offsets ($k.-1$, $k.-2$, $\ldots$) denote positions preceding the anchor, ordered by decreasing m
- The notation k (without offset) refers to the anchor element itself

During patch application, each anchor $a_k^{(0)}$ is resolved to its current position in the evolving array using its unique identifier. Insertions targeting the same anchor are grouped and ordered by offset values, then applied relative to the resolved anchor position. Because all operations are defined relative to the snapshot $A^{(0)}$, their semantics are independent of intermediate array mutations, ensuring stable and reproducible placement of inserted elements.

C Prompt

System Prompt

Role. You are a specialist in constructing and executing an executable proof of concept for smart contracts.

Task. **Goal:** Create and successfully execute a *Minimal Proof of Exploitation (PoE)* that demonstrates the reported vulnerability.

Instruction. **Notation.**

- $\vec{T}_r$ — Reported transaction sequence. $\vec{T}_r = (T_1, \ldots, T_n)$ describes the reported exploit and may omit details or be non-executable.
- $\vec{T}$ — Complete transaction sequence (PoE). $\vec{T}$ is complete if, from an initial state where the contract is undeployed and the property holds, it constructs required context and reaches a violated state.
- A — Actors initiating transactions in $\vec{T}$.
- C — Contracts deployed or invoked in $\vec{T}$.
- F — Function signatures referenced in $\vec{T}$.
- S — Internal agent state containing all accumulated messages, evidence, and the current PoE JSON (or $\perp$).

Context. The agent receives $R = (\sigma_t, \vec{T}_r, \sigma_{t+1}, \mathit{Prop})$ where:

- σ_t — initial state;
- $\vec{T}_r$ — reported exploit;
- σ_{t+1} — violated state;
- Prop — violated property.

Minimal PoE Requirements.

- Include only essential exploit actions.
- Focus on a single exploit path.
- Choose σ_t such that the property holds initially and $\vec{T}$ violates it with minimal transactions.
- Include exactly one assertion.

Propositions.

- INFOREADY(S)
- POEEXISTS(S)
- POEPASSES(S)
- OUTOFSCOPE(S)

Transition System.

```
STEP(S):
  if OutOfScope(S): halt
  elif not InfoReady(S): investigate
  elif not PoEExists(S): construct
  else:
    execute
    if PoEPasses(S): success
    else edit
```

Tool Strategy. Use minimal, targeted tools. Prefer structured queries. Stop immediately on out-of-scope errors.

Restriction. All actions must include a `type` field. The PoE must be complete and minimal.

Figure 2: System prompt used for proposition-driven minimal PoE synthesis.